! pip install = ! this is used in google colab and jupyter notebook to run terminal cammand directly, Pip is python package manager it installs and manages the paython packagaes which are not included in python by default.

pydeck = It is a python library use to visualize geospatial data 3D, it powered by deck.gl is a GPU-powered framework for visual exploratory data analysis of large datasets. it is built on Deck.gl + Mapbox. Used in Jupyter, Streamlit, web dashboards Example layer types Scatterplot, Hexagon, Arc, Line, Path, Text, Heatmap.

```
! pip install pydeck
```

Show hidden output

Import Pandas = Which is powereful data analysis and data manipulation tool in python.

pd = it is an alias.

pdk = it used for interactive map and 3D data visualization.

```
import pandas as pd
import pydeck as pdk
```

Data_url = It is defining the data location from where can load the data into colab for visualization.

```
Data_url ="https://raw.githubusercontent.com/ajduberstein/geo_datasets/master/small_waterfall.csv"
```

pd = Which is refers as Pandas library

.read_csv = It is a pandas function which is reads a CSV file.

Here the above mentioned data_url file has been read

```
df = pd.read_csv(Data_url)
```

pdk.Layer() = creates a visualization layer — like a transparent sheet

data=df = The source of data points. It tells Pydeck to use the DataFrame df, which must have at least x, y, z columns (and optionally r, g, b for color). Each row in df represents one point in 3D space.

get_position=["x", "y", "z"] = Tells Pydeck which columns represent the 3D coordinates of each point.

x = longitude or X coordinate

y = latitude or Y coordinate

z = height or elevation

This is how the 3D scene is plotted in space.

get_color=["r", "g", "b"] = Each point will be colored using RGB values from the dataset. r, g, b are numeric values from 0-255. This makes it possible to color points based on their reflectance or classification.

get_normal=[0, 0, 15] = Defines the normal vector direction (like the direction light is hitting the points from). It affects how light and shadow appear on the 3D points, giving a more realistic look.

point_size=0.8 = Specifies how large each point should appear in the 3D view. as example Smaller values (e.g., 0.2) → finer, more detailed look Larger values (e.g., 2.0) → chunkier, denser appearance

pickable=True = Allows interaction with the map — where we can click or hover over a point to identify or highlight it.

pdk.ViewState() = Defines the camera's initial position and angle in 3D visualization — basically how the user first sees point cloud.

**target=[df.x.mean(), df.y.mean(), df.z.mean()] ** = This centers the camera's focus on the middle of teh data. df.x.mean(), df.y.mean(), and df.z.mean() calculate the average coordinates of all points, giving a natural center for the scene.

rotation_x=15

Tilts the camera downward by 15 degrees — this gives a slight 3D perspective rather than a flat top-down view.

rotation_orbit=30 = Rotates the camera around the scene by 30 degrees, creating a nice viewing angle.

zoom=0.8 = Controls how close or far the camera starts.

Higher zoom → closer

Lower zoom → farther

controller= True

Enables interactive control = so we can drag, zoom, or rotate the view with your mouse.

Defining the 3D Orbit View view3d = pdk.View(type="OrbitView", controller=True)

pdk.View() = defines how the user interacts with the scene.

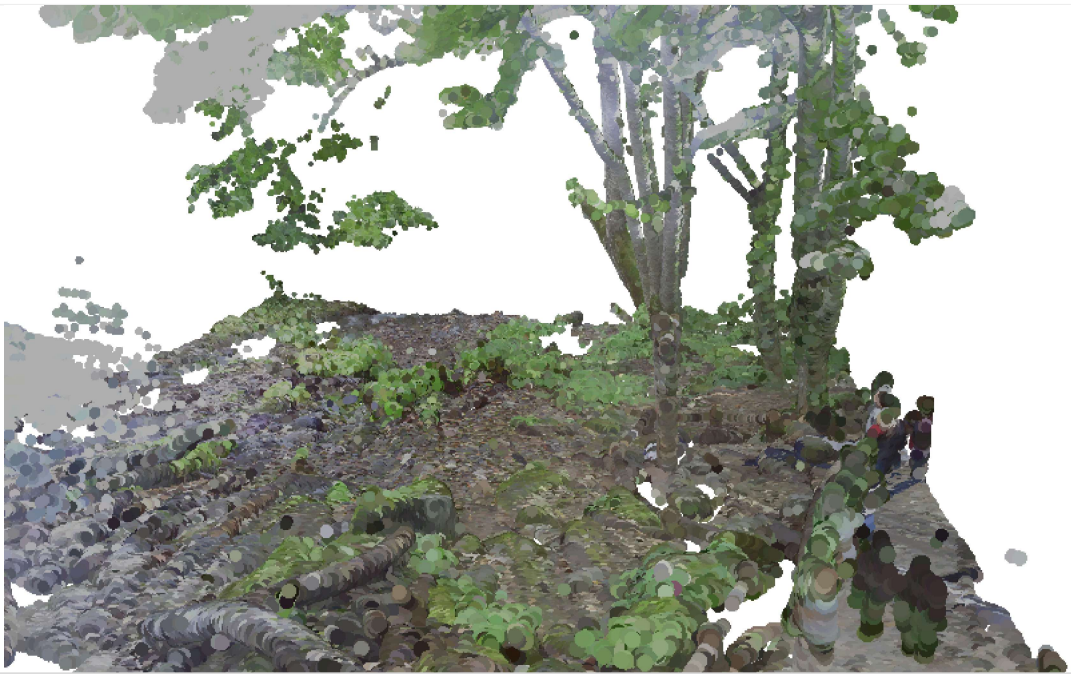
**type="OrbitView" ** = enables 3D orbit control, where you can freely rotate around the point cloud — ideal for LiDAR or 3D visualization.

controller=True again ensures interactive movement.

```
point_cloud = pdk.Layer(
    "PointCloudLayer",
    data=df,
    get_position=["x", "y", "z"],
    get_color=["r", "g", "b"],
    get_normal=[0, 0, 15],
    point_size=0.8,
    pickable=True,
    auto_highlight=True,
)

view = pdk.ViewState(
    target=[df.x.mean(), df.y.mean(), df.z.mean()],
    rotation_x=15,
    rotation_orbit=30,
    zoom=5,
    controller=True,
)

view3d = pdk.View(type="OrbitView", controller=True)
deck = pdk.Deck(layers=[point_cloud], initial_view_state=view, views=[view3d])
deck.show()
```



deck.to_html("point_cloud.html")

deck → This is the Pydeck visualization object (created earlier with pdk.Deck()).

.to_html() → This method exports the interactive visualization into a standalone HTML file.

Output_path = os.path.abspath("point_cloud.html")

**os.path.abspath() → ** From Python's built-in os module.

It converts a relative path (like "point_cloud.html") into an absolute path, showing the full directory location where the file is saved.

output_path The full path of the HTML file notebook_display=False Prevents automatic display in Jupyter Notebook after saving

```
# Export to an interactive HTML file
deck.to_html("point_cloud.html")


output_path = os.path.abspath("point_cloud.html")
deck.to_html(output_path, notebook_display=False)
print(f" Exported HTML file saved at:\n{output_path}")
```